REACT & REDUX

Interview Preparation Guide

Core Concepts • Hooks • Patterns • Redux Toolkit • Diagrams • Rapid-Fire Q&A

Prepared for Dipan Pramanik — MERN Stack Developer
Netaji Subhash Engineering College, Kolkata

Table of Contents

1 React Fundamentals

Why React?

- **Declarative, not Imperative** — plain JS requires step-by-step DOM instructions; React only needs the desired end state.
- **Reusability** — UI is built from small, composable components.
- **Readability & Maintainability** — component-based structure scales better for large apps.

Core Vocabulary

- **SPA (Single Page Application)** — one HTML shell; JS swaps views without full page reloads.
- **Alternatives** — Angular, Vue.
- **Component** — a self-contained, reusable piece of UI code.
- **className** — used instead of `class` because `class` is a reserved JS keyword.
- **JSX expressions** — embed JS inside markup with `{ variable / logic }`.

Props vs State

- **Props** — pass data from parent → child (read-only from the child's side).
- **props.children** — whatever content is nested inside a component's tags.
- **State** — a variable owned by a component that, when updated, re-renders the UI.
- **Child → Parent communication** — parent defines a function, passes it down as a prop, child calls it to "talk back" up.
- **key** in `.map()` — a stable unique identifier so React can efficiently track, reuse and update list items across renders.

2 Virtual DOM & Reconciliation

The **Virtual DOM** is a lightweight in-memory JS representation of the real DOM. Direct real-DOM manipulation is expensive, so React updates the Virtual DOM first, **diffs** it against the previous version, and applies only the minimal set of changes to the real DOM — a process called **Reconciliation**.

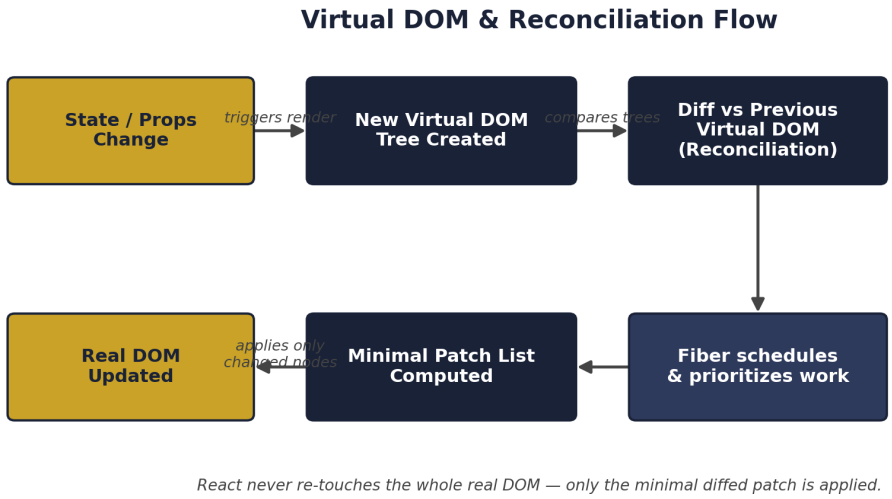

Fig 1. State change → new Virtual DOM → diff → minimal real-DOM patch.

Diffing Rules React Assumes (for speed)

- Different element types (e.g. `<div>` vs `<span>`) → old tree destroyed, new tree built from scratch.
- Same element type → DOM node is kept; only changed attributes/props are updated.
- Lists → matched across renders using `key`, not index position.

React Fiber

Fiber (React 16+) is the reconciliation engine that breaks rendering work into small interruptible units. It lets React pause, abort, or resume work, assign priority to different updates, and power concurrent features like Suspense and Transitions. The older React 15 "Stack Reconciler" was synchronous and blocking.

Why NOT use array index as a key?

If list order changes (insert / delete / reorder), index-based keys get reused for *different* items — React misidentifies elements, causing wrong DOM reuse, stale state inside list items, and broken animations. Always use a stable unique id from the data (e.g. `item._id`) unless the list is static and never reorders.

3 Hooks — Deep Dive

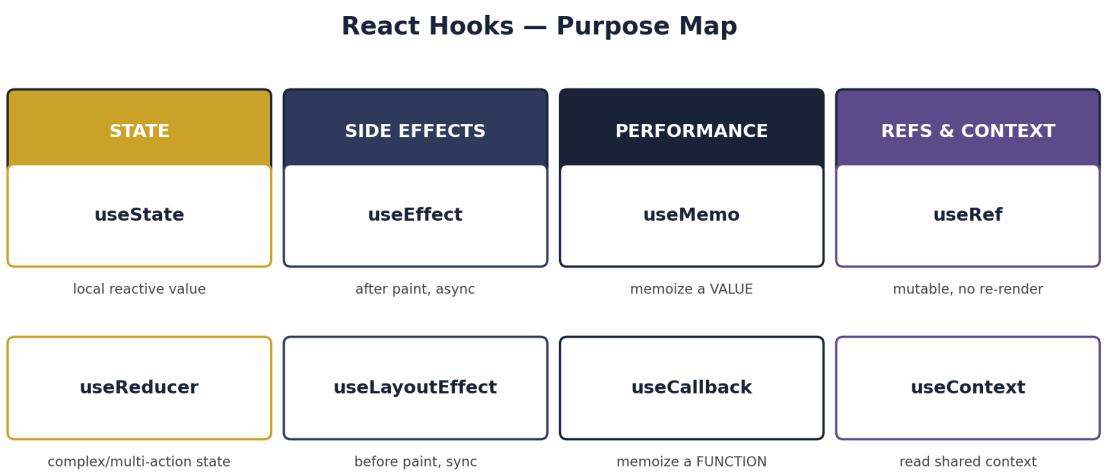

Fig 2. Hooks grouped by purpose.

useState

```
const [state, setState] = useState(initialValue);
setState(newValue);
setState(prev => prev + 1); // functional update
```

Creates and manages local state. Updating state schedules a re-render. Use the functional-update form when the next value depends on the previous one.

useEffect

```
useEffect(() => { /* runs after every render */ });
useEffect(() => { /* runs once on mount */ }, []);
useEffect(() => { /* runs on mount + when value changes */ }, [value]);
useEffect(() => {
// effect logic
return () => { /* cleanup before re-run / unmount */ };
}, []);
```

Handles side effects (data fetching, subscriptions, timers). The dependency array controls when it re-runs; an empty array runs once on mount.

useMemo vs useCallback

- **useMemo** — memoizes a computed **value**. `const v = useMemo(() => heavyCalc(a,b), [a,b])`
- **useCallback** — memoizes a **function** reference. `const fn = useCallback(() => doX(id), [id])`
- Both exist mainly to preserve referential equality so `React.memo` children don't re-render unnecessarily. Don't overuse — memoization has its own cost.

useRef

- Returns a mutable `{ current }` object that persists across renders **without** causing a re-render.
- Use 1: direct DOM access — `inputRef.current.focus()`.
- Use 2: storing mutable values (timer ids, previous values, render counts) that shouldn't trigger re-render.
- **useState vs useRef** — useState is reactive (triggers render); useRef is silent (no render).

useReducer

```
const [state, dispatch] = useReducer(reducer, initialState);
function reducer(state, action) {
switch (action.type) {
case "increment": return { ...state, count: state.count + 1 };
default: return state;
}
}
dispatch({ type: "increment" });
```

Preferred over useState when state logic is complex, involves multiple related sub-values, or the next state depends heavily on the previous one. *Interview one-liner: useReducer is basically Redux inside a single component.*

useContext

Reads a Context value directly: `const value = useContext(MyContext)`. Must be used under the matching Provider. Caveat: any change to the context value re-renders **all** consuming components — for large / frequently-changing global state, prefer Redux.

Rules of Hooks

- Only call hooks at the **top level** of a function component (never inside loops, conditions, or nested functions).
- Only call hooks from React function components or other custom hooks.
- Why: React tracks hooks by **call order** internally — breaking order breaks state association across renders.

4 Component Patterns & Advanced APIs

Custom Hooks

A JS function starting with `use` that internally calls other hooks to encapsulate reusable stateful logic (e.g. `useFetch(url)`, `useDebounce(value, delay)`). Cleaner than HOCs/render props — no wrapper-hell, easier to test.

Higher-Order Component (HOC)

```
const withLogger = (Component) => (props) => {
// cross-cutting logic
return <Component {...props} />;
};
```

A function that takes a component and returns an enhanced one. Classic example: `connect()` from react-redux. Mostly replaced by custom hooks today, but still asked in interviews.

Render Props

A pattern where a prop is a function returning JSX: `<DataProvider render={(data) => <Table data={data}/>} />`. Also largely superseded by hooks.

Error Boundaries

Class components implementing `static getDerivedStateFromError()` and/or `componentDidCatch()` to catch render-time errors in their child tree and show a fallback UI. **No hook equivalent exists** — must be a class component. They do **not** catch: event-handler errors, async code, or SSR errors.

Portals, Fragments, forwardRef

- **Portals** — `createPortal(child, domNode)` renders children outside the parent DOM hierarchy (modals, tooltips) while keeping React context/event bubbling intact.
- **Fragments** — `<>...</>` groups children without an extra DOM node.
- **forwardRef** — lets a parent pass a `ref` down into a child's DOM node, since function components don't accept `ref` as a normal prop.
- **useImperativeHandle** — used with forwardRef to expose a custom API (e.g. only `focus()`, `clear()`) instead of the raw DOM node.

useLayoutEffect vs useEffect

- **useEffect** — runs asynchronously, *after* the browser paints.
- **useLayoutEffect** — runs synchronously, *before* paint; blocks visual update until done. Use only when you must measure/mutate the DOM before the user sees a flicker.

Class Lifecycle ↔ Hooks Mapping

Phase	Class Lifecycle	Hook Equivalent
Mount	constructor → render → componentDidMount	useEffect(() => {...}, [])
Update	render → componentDidUpdate	useEffect(() => {...}, [deps])

Unmount	componentWillUnmount	useEffect cleanup function
Error	componentDidCatch / getDerivedStateFromError	No hook equivalent (Error Boundary)

PureComponent, Controlled vs Uncontrolled

- **PureComponent** — class-component equivalent of `React.memo`; auto shallow-compares props/state.
- **Controlled input** — value driven by React state (`value` + `onChange`).
- **Uncontrolled input** — DOM manages its own value; accessed via `useRef` (use for file inputs — files can't be set programmatically).

5 Forms & Controlled Components

Q: What is a Controlled Component?
A: An input whose value is completely controlled by React state.

Q: How do you handle multiple form fields with one state object?
A: Use the input's `name` as a dynamic key: `setFormData(prev => ({ ...prev, [name]: value }))`. The spread preserves existing fields; the computed key updates only the current field.

Q: Why use `checked` instead of `value` for a checkbox?
A: Checkbox state is boolean, so: `type === "checkbox" ? checked : value`.

Q: How do you manage radio buttons?
A: Give them the same `name` and store the selected value in state; `checked={formData.mode === "Online"}` lets state drive the UI.

Q: How do you make a <select> a controlled component?
A: Use `value={formData.favCar}` and update it through `onChange`.

Q: Why call `e.preventDefault()` on form submit?
A: It stops the browser's default page reload and lets React handle the submission.

Q: Why doesn't `console.log(formData)` show the new value right after `setFormData`?
A: State updates are scheduled/asynchronous; the current function closure still sees the previous state.

Interview one-liner: Controlled form = State → Input value → User change → onChange → State update → Re-render.

6 React Router

A library for client-side navigation between pages/components without reloading the whole page.

API	Purpose
`BrowserRouter`	Enables client-side routing; syncs UI with the browser URL.
`Routes / Route`	Routes selects the best matching Route; Route maps a URL path to a component.
`Link`	Client-side navigation without full page reload (vs a plain <a> tag).
`Outlet`	Renders the matched child route's element inside a parent layout route.
`useNavigate()`	Programmatic navigation: navigate("/about"), navigate(-1) back, navigate(1) forward.
`useLocation()`	Reads current URL info: pathname, search, hash, state.
`useSearchParams()`	Reads/writes query params, e.g. ?page=2&category=tech.
`path="*"`	Catch-all route used to render a 404 / Not Found page.

Flow: BrowserRouter → Routes → Route → Component. Link/useNavigate handle navigation; Outlet renders nested child routes; "*" catches unmatched URLs.

Context API (avoiding Prop Drilling)

Prop Drilling is passing data through many intermediate components just to reach a deeply nested child. **Context API** is React's built-in fix: create context → create a Provider → store shared state in it → wrap the needed tree → consume with `useContext()` in any descendant, however deep.

7 React 18 & Performance

React 18 Headline Features

- **Automatic Batching** — multiple setState calls (even inside promises/timeouts/native handlers) batch into one re-render, not just inside React event handlers as in React 17.
- **Concurrent Rendering** — React can prepare multiple UI versions simultaneously and interrupt low-priority renders for urgent updates.
- **useTransition** — marks updates as low priority so the UI stays responsive: `const [isPending, startTransition] = useTransition()`.
- **useDeferredValue** — defers re-rendering a non-critical part of the UI (great for search-as-you-type).
- **useId** — generates stable unique IDs matching on server & client (avoids SSR hydration mismatch).
- **createRoot** API replaces `ReactDOM.render`; plus streaming SSR & Suspense improvements.

Code Splitting / Lazy Loading

```
const About = React.lazy(() => import('./About'));
<Suspense fallback={<Loader/>}>
<About />
</Suspense>
```

Splits the JS bundle so components load only when needed; Suspense shows a fallback while the chunk loads.

Strict Mode

`<React.StrictMode>` is a dev-only tool that intentionally double-invokes certain functions (render, state initializers, effects) to surface impure/side-effect bugs early. It has no effect in production.

Performance Optimization Checklist

- `React.memo` for pure functional components.
- `useMemo` / `useCallback` to preserve referential equality.
- Proper unique `key` for lists — never index for dynamic lists.
- Code-split with `React.lazy` + Suspense.
- Virtualize long lists (react-window / react-virtualized).
- Avoid inline object/array/function literals as props (breaks memoization).
- Debounce/throttle expensive handlers (search input, scroll).
- Colocate state close to where it's used instead of over-globalizing it.
- Split contexts / use a state library for frequently-changing global data.

8 Redux — Fundamentals

Redux is a predictable **state management library** that centralizes all application state in one store, solving prop drilling and scattered/uncoordinated state across many components.

Three Core Principles

- **Single Source of Truth** — the entire app state lives in one store (one JS object tree).
- **State is Read-Only** — the only way to change state is to dispatch an action; never mutate directly.
- **Changes via Pure Functions** — reducers are pure: `(prevState, action) => newState`, no side effects, no mutation.

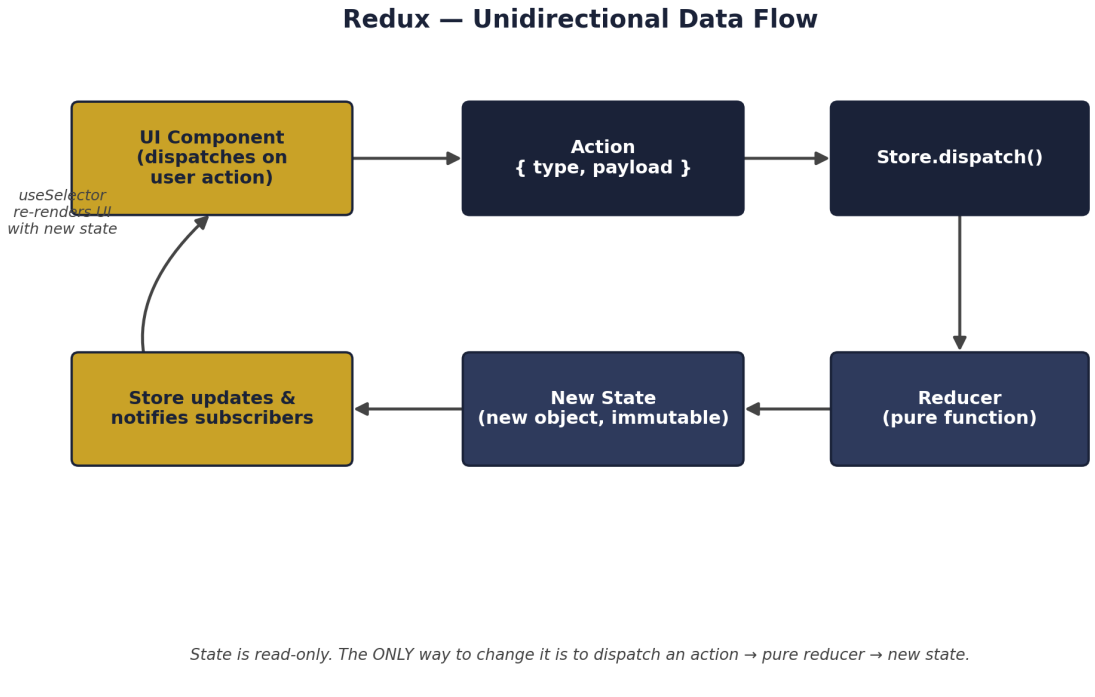

Fig 3. Redux's strict unidirectional data flow.

Core Building Blocks

Concept	Definition
Store	Holds the entire app state. store.getState(), store.dispatch(action), store.subscribe().
Action	Plain object describing "what happened". Requires a type field, optional payload.
Action Creator	Function returning an action object, e.g. addItem(item) => ({type, payload}).
Reducer	Pure function (state, action) => newState. Never mutates; always returns new state.
Dispatch	The only way to trigger a state change: store.dispatch(action).
Selector	Function that reads a specific slice of data from the store.

Why Immutability Matters

React/Redux detect changes via **reference** comparison (===), not deep comparison. Mutating state in place keeps the same reference, so nothing appears to have changed and no re-render happens. Immutability also enables Redux DevTools' **time-travel debugging**, since each action needs a distinct before/after state snapshot.

Redux vs Context API

- Context API re-renders **all** consumers on any value change — no fine-grained subscription; fine for rarely-changing global data (theme, auth user).
- Redux (via `useSelector`) re-renders only components whose selected state slice actually changed.
- Redux adds middleware, DevTools time-travel, and a strict unidirectional flow — better for large/complex apps with frequent async state changes.

9 Redux Toolkit (RTK)

Plain Redux required heavy boilerplate: separate action type constants, action creators, switch-case reducers, and manual spreads for immutability. **Redux Toolkit** is the official, opinionated toolset that removes nearly all of it and is now Redux's recommended standard.

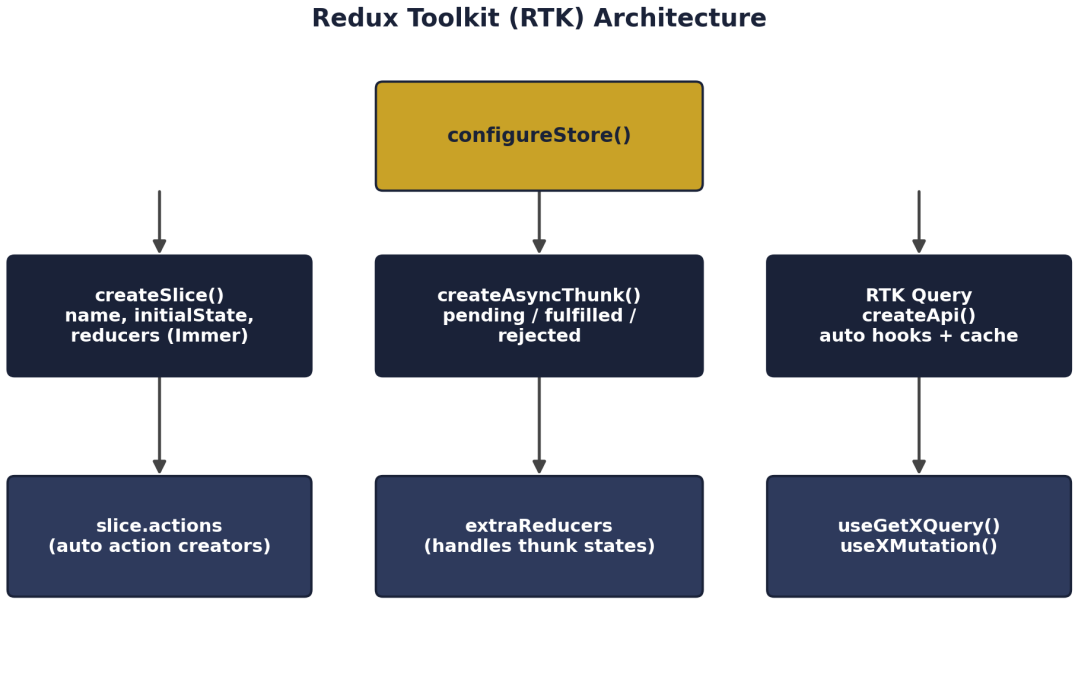

Fig 4. How RTK's pieces fit together.

configureStore

```
const store = configureStore({
reducer: { cart: cartSlice.reducer, user: userSlice.reducer }
});
```

Sets up the store with good defaults automatically: Redux DevTools enabled, redux-thunk included, dev-mode mutation/serializability checks.

createSlice

```
const cartSlice = createSlice({
name: "cart",
initialState: { items: [] },
reducers: {
addItem: (state, action) => { state.items.push(action.payload); }, // safe!
removeItem: (state, action) => {
state.items = state.items.filter(i => i.id !== action.payload);
}
}
});
export const { addItem, removeItem } = cartSlice.actions;
export default cartSlice.reducer;
```

Q: How can we "mutate" state directly inside createSlice reducers when Redux requires immutability?
A: RTK uses the **Immer** library internally. Immer lets you write code that looks like direct mutation on a "draft" state, but under the hood produces a brand-new immutable state object — the immutability rule is still respected, Immer just handles it for you.

createAsyncThunk

```
export const fetchUsers = createAsyncThunk("users/fetchUsers", async () => {
const res = await fetch("/api/users");
return res.json();
});

extraReducers: (builder) => {
builder
.addCase(fetchUsers.pending, (state) => { state.loading = true; })
.addCase(fetchUsers.fulfilled, (state, action) => {
state.loading = false; state.list = action.payload;
})
.addCase(fetchUsers.rejected, (state, action) => {
state.loading = false; state.error = action.error.message;
});
}
```

Automatically dispatches 3 lifecycle actions: `pending` (request started) → `fulfilled` (success) → `rejected` (failure).

Middleware: Thunk vs Saga

- **redux-thunk** (default, auto-included) — lets an action creator return a *function* receiving (dispatch, getState), enabling async logic. createAsyncThunk is built on top of it.
- **redux-saga** — uses generator functions for complex async flows (cancellation, debouncing, race conditions). More powerful, steeper learning curve.

react-redux Bindings

- `<Provider store={store}>` — makes the store available app-wide via context.
- `useSelector(state => state.cart.items)` — reads a slice of state; re-renders only when the selected value's reference changes.
- `useDispatch()` — returns the dispatch function to send actions.
- `connect()` — the legacy HOC used before hooks existed (still asked about in interviews).

Beyond the Basics

- **Redux DevTools** — inspects every dispatched action and state diff; supports time-travel debugging (auto-enabled by configureStore).
- **Normalized state** — store relational data as `{ byId, allIds }` instead of nested arrays for O(1) lookups; RTK's `createEntityAdapter` automates this.
- **RTK Query** — built-in data-fetching/caching layer; `createApi()` auto-generates hooks like `useGetUsersQuery()` with caching and cache invalidation via tags.

10 Interview Rapid-Fire Q&A

React

Q: Why is Virtual DOM fast?
A: Direct real-DOM writes are expensive; React diffs an in-memory Virtual DOM first and applies only the minimal real-DOM patch.

Q: useMemo vs useCallback in one line?
A: useMemo memoizes a computed value; useCallback memoizes a function reference.

Q: useEffect vs useLayoutEffect?
A: useEffect runs async after paint; useLayoutEffect runs sync before paint (use sparingly, only for DOM measurement).

Q: Why avoid index as a list key?
A: Reordering/inserting/deleting shifts indices, causing React to misidentify items and reuse the wrong DOM node/state.

Q: What can't Error Boundaries catch?
A: Event handler errors, async code errors, SSR errors, and errors inside the boundary itself.

Redux

Q: Is Redux only for React?
A: No — Redux is framework-agnostic; react-redux is just the React binding library.

Q: Can you have multiple Redux stores?
A: Technically possible but strongly discouraged; Redux is designed around one single source of truth.

Q: What happens when you dispatch an action with an unknown type?
A: Every reducer's default case returns the current state unchanged — no error is thrown.

Q: Can reducers have side effects (API calls, Date.now(), random values)?
A: No — reducers must be pure. Side effects belong in middleware/thunks such as createAsyncThunk.

Q: How does useSelector avoid unnecessary re-renders?
A: It subscribes to the store and re-renders only when the selected return value differs by reference from the previous render.

Q: What boilerplate does Redux Toolkit remove?
A: Classic Redux needed separate action type constants, action creators, and switch-based reducers with manual spreads; createSlice collapses all three using Immer for safe mutation syntax.

React = declarative UI via Virtual DOM diffing. Redux = predictable state via Action → Reducer (pure) → New State → UI. Redux Toolkit is the boilerplate-free, official way to write Redux today.